

Hybrid Biomedical Image-Analysis Pipeline: Stained-Nuclei Segmentation and LLM Reporting

Israel Ajayi — Data Science / AI module — Assignment 3

Modality: synthetic stained-nuclei fluorescence microscopy (Nickolay-K/Assignment-3-dataset).

1. Methods overview

The pipeline follows raw image → segmentation → region features → structured JSON → narrative, using three complementary methods rather than one: a multimodal LLM (llama3.2-vision) that reads the image directly, a classical pipeline (Otsu thresholding, morphological clean-up, connected-component labelling, regionprops) whose numeric output is handed to a text LLM (llama3.2), and a trained U-Net whose predicted mask feeds the same classical/LLM chain. The three methods are combined rather than compared in isolation because each addresses a different weakness of the others: Otsu and the U-Net are deterministic and auditable but blind to context; the VLM has visual context but no guarantee its claims are grounded; the numbers-first LLM step is grounded but only as good as the upstream segmentation.

All images were grayscale-converted and resized to 256×256. A standard luminosity conversion compresses this dataset's signal badly (max intensity $\approx 90/255$ versus $255/255$ in the raw blue/DAPI channel), so a 1st/99th-percentile contrast stretch was applied after grayscale conversion — documented in the notebook and reflected in the EDA figure below.

2. Task 1 — EDA and VLM description

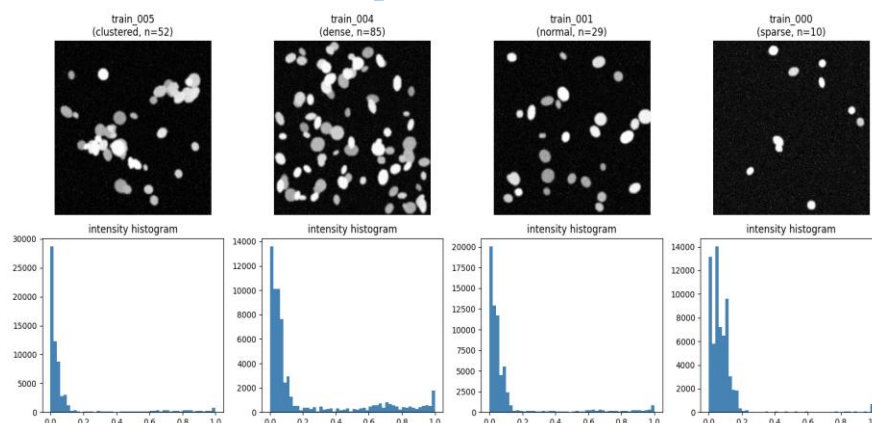


Figure 1. Sample images (one per density regime: sparse, normal, dense, clustered) with intensity histograms after grayscale conversion and contrast-stretch.

Optimised prompt

You are assisting with the documentation of a biomedical microscopy image for a research log. You are NOT a diagnostic tool: you must not offer any clinical opinion, diagnosis, or health recommendation... Respond with ONLY a JSON object: { "modality", "tissue_type", "notable_features", "image_quality" } If you cannot determine a field confidently from the image alone, set it to "uncertain". (Full prompt text in the notebook, Task 1 cell; representative image: train_004, dense, ground truth n=85.)

Naive prompt ("Describe this image.") — response

"The image presents a microscopic view of cells, characterized by a dark background and blue, irregularly shaped cells... The cells are not uniform in size or shape, with some being larger or more elongated than others... The blue color is intense and vibrant, making the cells stand out against the dark background... The darkness of the background helps to emphasize the shape and size of the cells, making them appear more prominent." (abridged; full text in the notebook, cell 11 output).

Engineered prompt — response (parsed JSON)

```
{ "modality": "fluorescence microscopy", "tissue_type": "cell culture",  
  "notable_features": "Blue, irregularly shaped objects are visible, likely  
    representing cells or cell structures.",
```

```
"image_quality": "good" }
```

Repeated-run variability (same image, same prompt, 3 calls)

All 3 runs were different (confirmed: `all_identical = False`). `tissue_type` varied across “cells” / “cell culture” / “cellular”; `notable_features` varied from “no apparent clustering” to “even distribution” to “bright blue dots, scattered” for the same image. `image_quality` was the only field stable across all three runs (“good”).

3. Task 2 — Classical features and numbers-first LLM

Otsu thresholding + regionprops on `train_004` (dense, ground truth `n=85`) found 45 connected components. The shortfall is not misdetection — it is that touching/overlapping nuclei merge into a single connected component under binary thresholding. This was confirmed to be a property of binary segmentation itself, not a weakness specific to Otsu: even the ground-truth binary mask for `train_005` (clustered, ground truth `n=52`) has only 17 connected components against 52 true instances in the 16-bit instance labels (Section 4).

Otsu's own summary of train_004, handed to the text LLM: “In train_004, 45 objects were detected. Object areas range from 48 to 1036 pixels (mean 294, median 222). Mean eccentricity is 0.73; mean solidity is 0.91. Mean object intensity is 0.75.”

Numbers-first LLM response (same image)

```
{ "n_objects": 45, "density_class": "sparse",  
  "shape_regularity": "highly irregular", "quality_flag": "ok" }  
"The detected objects are spread out across the image, indicating a sparse  
distribution. Their shapes exhibit highly irregular contours..."
```

Note the miscalibration: the model was given `n_objects=45` directly and still classified the image as “sparse” (ground truth density label: `dense`). This pattern recurs at scale in Task 4 (Section 5) and is discussed directly under Question 4/5 below — it is a different failure mode from hallucinating a number, and arguably a more dangerous one, because the number quoted is correct while the judgement built on it is not.

Answer — Question 1: which description is more useful / more trustworthy?

The VLM description is more useful when visual context matters beyond what regionprops measures — staining pattern, artefacts, overall field composition — because it sees the actual image; its naive-prompt response above does pick up genuinely visible detail (irregular shapes, overlapping cells, vivid blue-on-black contrast) that regionprops does not encode at all. It is less trustworthy in isolation: the repeated-run check shows its wording, and specifically substantive fields like `notable_features`, vary run to run for the same image at non-zero temperature. The numbers-first description is more trustworthy in the narrow sense that its `n_objects` is always the true regionprops count and it cannot invent a feature the measurements do not support — but Section 3's own result shows trustworthy inputs do not guarantee trustworthy judgements: the model saw `n_objects=45` and still output `density_class='sparse'`. In practice neither output should be trusted as-is: the VLM's role is best restricted to flagging things worth a human look, and the numbers-first LLM's categorical judgements (`density_class`, `shape_regularity`) need cross-checking against the numeric fields they are supposed to summarise, not just against the image.

4. Task 3 — U-Net training and evaluation

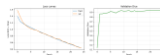


Figure 2. Training/validation loss (BCE+Dice) and validation Dice over 25 epochs.

Split / stage	n	Mean Dice	Mean IoU	Note
U-Net, val (all)	20	0.942	0.891	25 epochs, BCE+Dice loss, 128px, GPU runtime
U-Net, val, worst image (val_000, sparse)	1	0.872	0.773	lowest of all 20 validation images
U-Net, val, best image (val_005, clustered)	1	0.956	0.915	highest of all 20 validation images
Otsu, train_004 (dense, gt n=85)	-	-	-	45 connected components (undercounts)
GT binary mask, train_005 (clustered, gt n=52)	-	-	-	17 connected components (undercounts too)

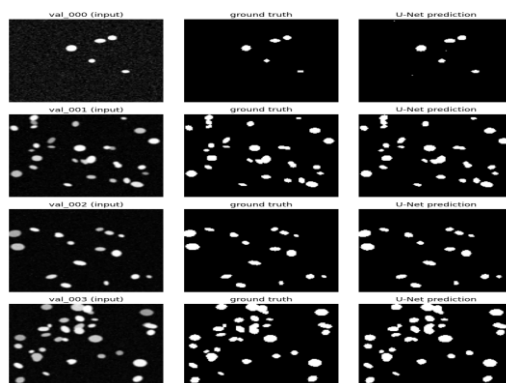


Figure 3. Input / ground truth / U-Net prediction for four validation images.

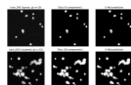


Figure 4. Otsu vs U-Net mask, one sparse and one clustered example.

Answer — Question 2: did the U-Net improve on Otsu?

At the mask level, yes: qualitatively the U-Net produces cleaner, less speckled foreground regions than Otsu, particularly on the sparse example, where Otsu's fixed global threshold either misses dim nuclei or picks up background speckle depending on the image's overall brightness, while the U-Net's learned features are more robust to this. On the clustered example, neither method separates touching nuclei into distinct components (see Section 3/4) — this is a shared limitation of binary segmentation, not something Otsu does better. One clear win for the U-Net: it needs no per-image threshold tuning, whereas Otsu's single global threshold is visibly unstable in mixed-brightness fields. One case where Otsu is arguably 'good enough' and cheaper: very sparse, high-contrast images, where a simple threshold already separates foreground and background almost perfectly and the extra model complexity buys little.

Answer — Question 3: Dice/IoU, and where the model makes mistakes

Final validation mean Dice = 0.942, mean IoU = 0.891 (n=20). Dice measures the overlap between predicted and ground-truth foreground ($2 \times \text{intersection} / (\text{prediction} + \text{ground-truth size})$); IoU is the stricter intersection-over-union of the same two regions. Both are close to 1 for a near-perfect mask and closer to 0 as prediction and ground truth diverge. The five worst-performing validation images were val_000, val_012, val_018 (all sparse, Dice 0.872–0.930) and val_009, val_013 (normal, Dice 0.937–0.940); the five best were val_004, val_016 (dense), val_008 (normal), and val_011, val_005 (clustered, Dice up to 0.956) — i.e. contrary to the intuition that dense/clustered images (harder-looking, more objects, more touching boundaries) would be where the model struggles most, the single worst score (val_000, 0.872) and two of the next three worst are sparse images, while dense and clustered images occupy most of the best-5 list. The reason is that Dice is a relative-overlap metric: a sparse image has very little total foreground area, so missing or over-segmenting even one or two small nuclei removes a large fraction of the (small) foreground, swinging the ratio sharply. The same absolute pixel error on a dense image, where the foreground area is much larger, barely moves the ratio. The practical implication is that Dice alone should not be read as 'the model fails on hard/crowded images' without checking the per-image breakdown — here it does not.

5. Task 4 — Hybrid pipeline on the unseen test set

The full pipeline (U-Net mask → regionprops → LLM JSON → narrative) was run on all 12 unseen test images and aggregated to hybrid_pipeline_report.csv. Each record carries an audit column, n_objects_measured, computed directly from regionprops — independent of anything the LLM claims — specifically so that a mismatch between the LLM's own n_objects field and n_objects_measured is a mechanical, checkable signal of drift or hallucination rather than something a reader has to judge by eye. Result: n_objects (LLM) matched n_objects_measured exactly on all 12/12 test images (e.g. test_002: 25 vs 25; test_004: 43 vs 43; test_010: 35 vs 35) — the model reliably copied the grounded count.

Example records

```
test_000: {"n_objects": 8, "mean_area": 52, "density_class": "sparse", "quality_flag": "ok"} (gt density: sparse, gt n=8)
test_004: {"n_objects": 43, "mean_area": 83, "density_class": "moderate", "quality_flag": "ok"} (gt density: dense, gt n=75)
test_005: {"n_objects": 15, "mean_area": 110, "density_class": "sparse", "quality_flag": "ok"} (gt density: clustered, gt n=49)
```

```
test_010: {"n_objects": 35, "mean_area": 96, "density_class": "sparse", "quality_flag": "ok"} (gt density:
dense, gt n=78)
```

The `n_objects`/`mean_area` fields are trustworthy (grounded, exact). `density_class` is not: it read “sparse” for 11 of the 12 test images and “moderate” for the twelfth, regardless of whether ground-truth density was sparse, normal, dense, or clustered, and regardless of `n_objects` ranging from 8 to 43 within the images it still called “sparse”. `quality_flag` was “ok” on all 12, including the two ground-truth “dense” test images with 75–78 true objects — it never once flagged an image for review. This is the single most important empirical result from Task 4: the audited, grounded fields are reliable; the model's own qualitative judgement on top of those same fields is not, and nothing in the JSON schema itself would tell a reader that without checking against `metadata.csv` as done here.

Answer — Question 4: where can the LLM hallucinate, and why does JSON-as-source-of-truth help?

The LLM can go wrong in (at least) two distinct ways, and this run produced clean evidence of both. First, classic hallucination: in Task 1, shown the raw image directly under a naive, unstructured prompt, it can describe detail not strictly verifiable from the image (the naive response's confident claim that “no visible nuclei or other internal structures” are present is itself a claim regionprops does not support one way or the other). Second, and more insidious because it does not look like hallucination at all: miscalibrated judgement on top of correct numbers. Every `n_objects` value in Task 4 was exactly right, yet `density_class` was essentially constant (“sparse”) and uncorrelated with either the true density label or the model's own `n_objects` field, and `quality_flag` never triggered even on the two worst-case dense images. Three design choices reduce the first failure mode: a fixed JSON schema, the explicit 'uncertain' escape hatch, and cross-checking any number against the deterministic upstream value. None of those three catch the second failure mode — catching it requires a further, independent check: comparing a categorical field like `density_class` against a simple deterministic rule computed from the same numbers the LLM was given (e.g. bucket `n_objects` into sparse/normal/dense/clustered thresholds directly), rather than trusting the LLM to apply that mapping correctly. This is why the JSON-as-source-of-truth principle has to extend beyond 'the LLM didn't invent a number' to 'the LLM's own labels are cross-checked against the numbers it was given.'

Answer — Question 5: would you trust any part of this system clinically, and what single change would most improve trustworthiness?

No part of this system should be trusted in a real clinical setting without a qualified human reviewer, and this is stated directly in the notebook at both the Task 1 and Task 4 markdown cells. The deterministic measurement layer (Otsu/regionprops, U-Net mask + regionprops) is the most trustworthy component because it is reproducible, auditable, and does not vary between runs — but it is not accurate enough on its own: it was shown directly (Section 3/4) that binary segmentation, whether from Otsu or the U-Net, cannot separate touching nuclei, and the ground-truth mask itself has this limitation. The LLM layer is the least trustworthy, and Task 4 shows this concretely: `quality_flag` stayed 'ok' on 12/12 images including two dense ones with 75+ true objects, and `density_class` tracked almost nothing about the actual image. The single change most likely to improve trustworthiness is a pre-LLM (or LLM-independent) quality/consistency gate — a cheap, deterministic check that (a) rejects or flags an image before the LLM is ever called if the measured features look extreme, and (b) recomputes `density_class`-style categorical fields directly from thresholds on the measured numbers rather than accepting the LLM's own labelling of them, using the LLM only for the free-text narrative layer humans read, never for the categorical judgement a downstream system might act on.

6. Extension — robustness trace

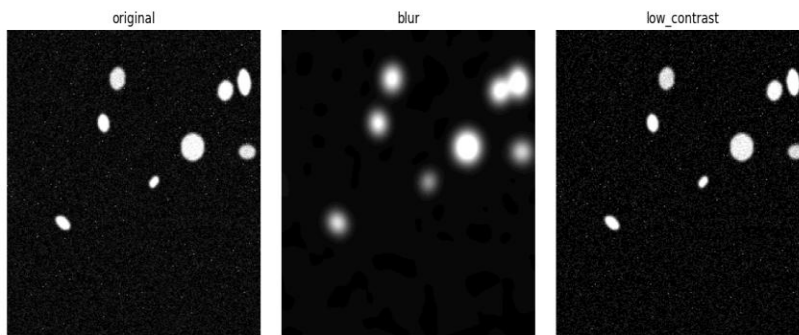


Figure 5. Original vs blurred vs low-contrast variants of test_000.

Variant	fg_pixels	n_objects	mean_area
original	418	8	52.25
blur	1197	7	171.00
low_contrast	411	8	51.38

Blur nearly tripled `fg_pixels` (418→1197) and more than tripled `mean_area` (52→171) while losing one detected object (8→7) — nuclei visually bleeding together into fewer, larger regions. This is detectable at Stage 1/2 (segmentation/feature extraction), before any LLM call, purely from the measured numbers. Low contrast barely shifted these numbers at all (418→411 `fg_pixels`, 52.25→51.38 `mean_area`), because the percentile contrast-stretch already applied during preprocessing (Section 1) neutralises it — a case where a design choice made for an unrelated reason (fixing the blue-channel grayscale compression) incidentally bought robustness against one corruption mode but not another. Given Section 5's finding that `quality_flag` never once triggered even on genuinely dense/crowded images, it should not be assumed the LLM step would catch the blur case either — the earliest reliable detection point remains the deterministic Stage 1/2 features, not the LLM.

References

Ronneberger, O., Fischer, P. and Brox, T. (2015) 'U-Net: Convolutional Networks for Biomedical Image Segmentation', MICCAI 2015.

Otsu, N. (1979) 'A Threshold Selection Method from Gray-Level Histograms', IEEE Trans. SMC.

van der Walt, S. et al. (2014) 'scikit-image: image processing in Python', PeerJ 2:e453.

Ollama (2024) local LLM runtime, <https://ollama.com> — models used: llama3.2, llama3.2-vision.